CTIP: Towards Accurate Tabular-to-Image Generation for Tire Footprint Generation

Daeyoung Roh, Donghee Han, Jihyun Nam
KAIST
dybroh, donghee.han, eliza1102@kaist.ac.kr

Jungsoo Oh, Youngbin You, Jeongheon Park
Hankook Tire & Technology
jungsoo, ybyou11, jeongheon@hankooktech.com

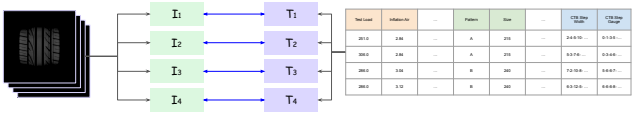
Mun Yong Yi
KAIST
munyi@kaist.ac.kr

Abstract

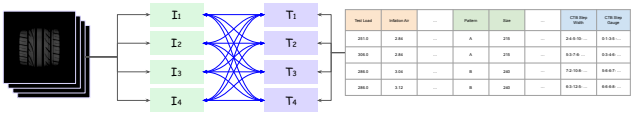
Generating images directly from tabular data while ensuring an accurate representation of ground truth is a useful application in manufacturing. Simply embedding tabular data to use it as a condition in image generation models often fails to learn the correspondence between tabular features and their impact on the generated image. To overcome this limitation, we propose ***C****ontrastive* ***T****abular-* ***I****mage* ***P****re-training (CTIP), inspired by the CLIP framework. These pre-train methods help improve the quality of the embedding of the tabular encoder on the tabular data, which then helps improve the performance of the image generation model. CTIP uses contrastive learning on multiple tabular and image data pairs, allowing the model to learn how changes in certain tabular features affect images. This approach is particularly crucial in manufacturing, where accurate capture of product outcomes under varying conditions is essential. We demonstrate that applying CTIP enhances image generation performance, yielding images that closely match ground truth images, even in Feature Few-shot or Feature Zero-shot scenarios where specific features are sparse or novel. We further show the application of CTIP in tire development, where tire footprint images are generated based on tire specifications and test conditions. CTIP produces high-quality embeddings that align well with ground truth images and effectively handle the scarcity or sparseness of specific features, addressing common challenges in new product development. Our code is available in* https://github.com/Noverse0/CTIP.git.

1. Introduction

Despite recent advancements in image generation models, their practical application remains limited, particularly in the manufacturing industry. While there have been studies under various conditions, such as text-to-image [17, 27, 35, 37, 39] and image-to-image translation [11, 12, 24], research specifically focused on tabular-to-image generation using tabular data is still scarce. However, tabular-to-image studies, where tabular data is used as a condition, hold significant potential in manufacturing, especially in areas such as new product development and product testing. This approach allows manufacturers to test products without needing physical prototypes, which can significantly speed up the development process, reduce costs, and improve environmental concerns caused by prototypes. To the best of our knowledge, this is the first tabular-to-image generation study that uses tabular data as a condition to generate precisely targeted images.



Training an image generation model: Single data pair.



Training CTIP: Multiple data pair.

Figure 1. Pre-training with CTIP can learn how feature changes affect an image through contrastive learning on multiple tabular data and images.

We have found limitations to simply embedding tabular data and using it as a condition in an image generation model. This approach struggles to learn how changes in

specific tabular features affect the image because it only trains on a single pair between the tabular data and the image. This problem necessitates prior learning that integrates visual information to understand the impact of feature variations in tabular data on images. To address this issue, we introduce a novel approach, **C**ontrastive **T**abular-**I**mage **P**re-training (CTIP), inspired by the successes of CLIP. Because tabular data in manufacturing consists of many features and the features are interdependent, it is challenging to reflect changes in a particular feature in an image precisely. Therefore, CTIP uses contrastive learning of multiple pairs of tabular and image data to train a tabular encoder on how changes in a particular tabular feature are reflected in an image. This pre-training method makes the tabular data more sensitive to capturing changes in the tabular feature, which is critical in manufacturing, where products are often tested under various conditions, and it is essential to produce desired results accurately. Figure 1 illustrates the advantages of using CTIP.

We demonstrate the application of image generation models in tire development, highlighting the practicality of using CTIP in the manufacturing industry. Unlike traditional image generation studies that aim to generate realistic images, we evaluate image generation performance by comparing it to ground truth (GT) images. We assessed the results using three tire performance metrics for the tire's shape and three similarity metrics with the GT image. Directly targeting and predicting tire performance metrics using tabular data does not explain the cause of the resulting values. For example, from the tabular data, users cannot see the resulting image, which makes it challenging to improve the problem because the current cause is unknown. Further, it takes a large amount of time, not to mention money and effort, to produce an actual tire based on the taular data. Therefore, generating tire images that match the given tabular data is an innovative and useful application because it permits checking the visual results that match the tabular data, without producing actual products.

We demonstrate that applying CTIP enhances image generation performance. Both generative adversarial neural networks (GANs) [2] and diffusion [36]-based models showed improved results when CTIP was utilized. Furthermore, when tabular data is used as a condition in an image generation model without pre-training, the generated images often deviate from the GT image, especially when information about a specific feature is sparse in tabular data. We define these scenarios as Feature Few-shot or Feature Zero-shot, indicating very few or no specific set of feature values are available in the training data. Even in these cases, CTIP robustly generates images, suggesting that CTIP produces high-quality embeddings even under extreme scenarios. Without CTIP, the model does not capture changes in the image when test conditions change, producing almost identical images across different test conditions. CTIP sensitively detects changes in test conditions and produces accurate images under different test scenarios. As a result, CTIP improves the quality of embedding generation for tabular data, contributing to better performance of image generation models and enabling robust image generation under hard cases.

Our main contributions are summarized as follows:

- We introduce a novel method, called CTIP (Contrastive Tabular- Image Pre-training), which applies the CLIP framework to pre-training multiple pairs of tabular data and images, improving a model's ability to produce accurate images from tabular data and presenting an example of its application to real-world manufacturing data.

- We show that CTIP effectively handles Feature Few-shot and Feature Zero-shot scenarios, generating high-quality and accurate images even when specific features are sparse or novel, addressing common challenges in new product development.

- We demonstrate that CTIP generates high-quality embeddings that are more similar to the actual image information compared to simply embedding tabular data. Additionally, CTIP captures changes in specific features more sensitively and accurately.

2. Related Work

Applications of CLIP CLIP has demonstrated exceptional performance in estimating semantic similarity between text and visual modalities, making it valuable in various downstream applications, from image classification to image generation tasks. In research on image generation models, CLIP has been shown to produce higher-quality images by more effectively embedding prompt information , with methods explored to leverage pre-trained CLIP models for efficient learning, even in data-sparse environments [23, 26, 39]. Furthermore, research in fields such as image manipulation [14, 29, 30, 32], text-to-video generation [1, 16, 34], audio methods [3, 18, 33], and others highlights CLIP's broad applicability across various domains. Despite this versatility, the direct application of CLIP to industrial contexts, particularly in handling tabular data, has been underexplored. Therefore, we aim to leverage CLIP's multi-modal learning framework to address the challenge of generating images from tabular data, a standard yet challenging format in the manufacturing industry.

Applications of CLIP with Tabular While not extensively explored, the application of CLIP to tabular data has recently begun to attract attention. Tang et al. [25] propose a study to generate images using tabular data and

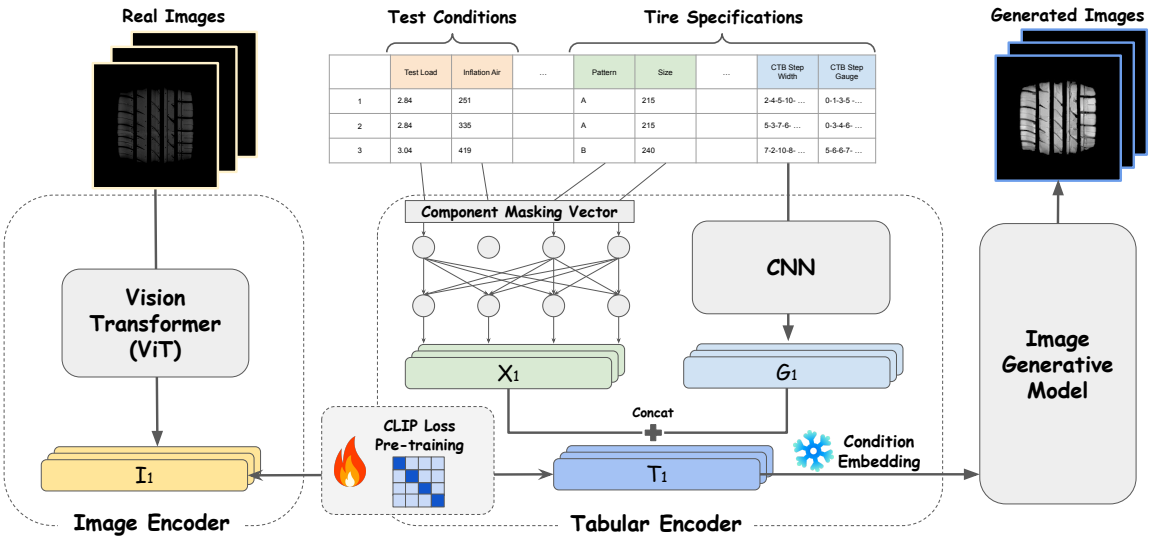


Figure 2. Illustration of the CTIP framework. We use clip loss to pre-train a tabular encoder and an image encoder. The tabular encoder is utilized to embed the tabular data. After pre-training, when training the image generation model, we freeze the weights of the tabular encoder. The image encoder is only used for pre-training with the tabular encoder.

present a benchmark dataset; however, their goal is to generate generic images using tabular data, whereas our goal is to generate precisely targeted images. Most existing research on CLIP investigates the use of CLIP loss on tabular data. For example, Hager et al. [4] researched predicting coronary artery disease (CAD) using CLIP loss with tabular data, including 120 clinical features and cardiac MR images in a multi-modal approach to predict CAD. Similarly, Huang, W. [7] explored the prediction of Alzheimer's disease by applying CLIP loss to neuroimaging and tabular data. However, our research differs from these multi-modal methods in that it aims to generate targeted images based on tabular data alone. To achieve this goal, both a tabular coder and an image coder were pre-trained, and then the tabular coder was used for image generation in our study.

Image Generation Models The development of image generation models has predominantly evolved around two main frameworks: GANs and diffusion models. GANs have made significant advances in generative modeling, most notably DF-GAN [27], DCGAN [20], and StyleGAN [8–10]. DF-GAN employs a novel target-aware discriminator that integrates matching-aware gradient penalty and one-way output mechanisms. This discriminator design improves text-image semantic consistency more effectively than previous methods that relied on auxiliary networks, thus simplifying the overall network architecture while enhancing its supervisory capabilities. Latent Diffusion Models (LDMs) [21] represent another innovative class of generative models that use diffusion processes within a learned latent space, offering a more efficient approach to generating high-dimensional data compared to traditional GANs and VAEs [13]. LDMs refine data through a series of stochastic processes, similar to physical diffusion, which has proven effective in generating high-quality images, text, and other data types.

3. Methodology

In this session, we will introduce CTIP, a pre-trained method for generating accurate and robust footprint images of tires using tabular data about tires. CTIP comprises two encoders: a tabular encoder and an image encoder. After pre-training the two encoders using clip loss, the image generation model is trained. Figure 2 illustrates the framework of CTIP and the training process of the image generation model.

3.1. CTIP Encoders

CTIP encoders consist of a tabular encoder and an image encoder. The tabular encoder processes tabular data, including tire specifications and test conditions, and generates a condition embedding. This embedding is then used as a condition in the image generation model. Additionally, an image encoder is used to generate a tire footprint image embedding that matches the dimensionality of the conditioned embedding. The image encoder is used during the pre-train process but is not involved in the image generation model training or inference phase.

3.1.1 Tabular Encoder

The tabular data used to generate the tire footprint includes features that provide tire specification information and test condition features from the tire experiments. We concatenate the tire specification features with the test condition features. These features fall into three categories: Numerical, Categorical, and Graphical. The graphical variable represents cross-sectional information about certain parts of the tire. Because an image could replace this graphical variable in other domains, it introduces the possibility of multimodal applications.

Numerical features are standardized for preprocessing using z-score normalization to enhance learning efficiency, and categorical features are converted into one-hot vectors. Numerical and categorical features are then passed through a component masking vector and scaled to the required dimensionality using a 2-layer MLP. Component masking vectors ensure that only relevant information for the tire is utilized. For instance, a tire may use one, two, or even three belt parts - in scenarios where only one belt is used, the features representing information about the second and third belts hold arbitrary values. To address this issue, masking vectors retain only the variable information for the components used in that specific tire. The preprocessing steps are outlined as follows:

$$X' = MLP(M \odot [X_{num}; X_{cat}])$$

where M is the structure masking vector, X_{num} the matrix of numerical features, and X_{cat} the matrix of categorical features.

Graphical features that contain location data and their values are processed uniquely. A transition in location value is denoted by a '-', which combines position and value information (e.g., '4-5-7-10' with '0-1-3-5') to represent value changes across specified positions. Thus, plotting values with position information transforms them into graphical features. Additional detail and example about graphical features can be found in the supplemental material. To preprocess these features, a convolutional neural network (CNN) compresses the information into a single dimension. The utilization of graphical features has the potential to evolve into a multi-modal that integrates tabular data and images. The preprocessing of graphical features is represented as follows.

$$G' = CNN(X_{graph})$$

After preprocessing, the processed numerical, categorical, and graphical feature embeddings are concatenated to form condition embeddings:

$$T = [X'; G']$$

3.1.2 Image Encoder

The image encoder is composed of a vision transformer (ViT). ViTs have the advantage of being able to divide an image into patches and learn the overall relationship between patches through the transformer architecture. Thus, they generally have a better understanding of the overall context than CNNs. The image encoder embeds the tire footprint image into an embedding of the same size as the condition embedding, which enables the integration of visual information and tabular data through contrastive learning. Using a ViT, tire footprint image embeddings I are generated from the footprint image img:

$$I = ViT(img)$$

Both T and I are simultaneously trained using CLIP loss to refine the quality of the condition embeddings and enable the image generation model to produce accurate tire footprint images.

3.2. Pre-training with CLIP Loss

For the two generated embeddings, T (tabular) and I (image), we apply contrastive learning using CLIP loss. In contrastive learning, the embeddings T and I, generated in batches, are multiplied to create a similarity matrix. In this matrix, the diagonal elements represent pairs of the same tabular data and image, while the off-diagonal elements represent pairs of different tabular data and images. The loss is calculated by treating the diagonal elements as positive pairs and the off-diagonal elements as negative pairs.

The loss function for contrastive learning can be expressed as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left(\log\frac{e^{\text{sim}(T_i,I_i)/\tau}}{\sum_{j=1}^{N} e^{\text{sim}(T_i,I_j)/\tau}}\right),$$

where $\text{sim}(T_i, I_i)$ represents the similarity between the i-th tabular embedding and its corresponding image embedding, τ is a temperature parameter, and N is the batch size.

This learning approach enables the tabular encoder to integrate information from both the tabular data and the images. As a result, it learns how changes in the tabular data affect the tire footprint image, making the model sensitive to variations in the tabular data and capable of generating high-quality condition embeddings.

3.3. Image Generative Models

CTIP is used to pre-train a tabular encoder, which is then used to train the image generation model. At this point, the weights of the tabular encoder are frozen. We use a GAN-based model and a diffusion-based model to validate CTIP. The GAN-based approach leverages DF-GAN [27], where conditional embeddings are used as input to generate target

tire footprint images. For diffusion, we adopt LDM [21], which utilizes pre-trained VAE weights and further trains with DDPM [5] structured with U-net [22].

4. Experiment

4.1. Dataset

A total of 64,756 tire footprint images and tabular data containing specification information about tires and test conditions were used. The tire footprint image resolution is 256×256. All tire footprint images are grayscale with one channel. The tabular data includes specification information about the tires and test condition information, serving as the conditions for generating the tire footprint images. There are a total of 116 features in the tabular data, including 113 pieces of tire specification information and 3 pieces of test condition information.

For this study, we simulated the process of developing a new tire by separating training and testing data based on tire specification information. There were 7,715 tire specifications in total, and to evaluate the results for each of the 191 tire patterns, one tire pattern specification was randomly assigned to a test set. In other words, all tires with the same specification were divided into training or test sets. Generally, we conducted experiments under different test conditions for one specification, with an average of 8.39 tests per tire specification and a median of 5.00. We defined a feature few-shot and zero-shot case for the tire pattern in the test set. A Feature Few-shot occurs when fewer than five specifications in the training set use that pattern, and a Feature Zero-shot is for a completely new pattern. For feature few-shot, there were 578 tire specifications; for feature zero-shot, there were 139 tire specifications. Additional dataset details can be found in the supplementary material.

4.2. Evaluation Metric

We used both tire performance and image similarity metrics to evaluate the generated tire footprint images. The tire performance metrics consist of Contact Length (CL), Contact Width (CW), and Contact Area (CA). These metrics assess the performance of the tire using the actual tire footprint. Mean Absolute Percentage Error (MAPE) was used to compare the predicted values from the generated footprint images against the GT images obtained from the actual products. An additional explanation of the Tire Performance matrices can be found in the supplemental material.

We also used image similarity metrics to compare the generated images against GT images. These metrics include Learned Perceptual Image Patch Similarity (LPIPS) [38], Structural Similarity Index (SSIM) [31], and Peak Signal-to-Noise Ratio (PSNR) [6]. For generated tire footprint images, background noise was removed before conducting similarity comparisons. Although these metrics may not ideally compare images due to intrinsic differences, they align closely with human visual evaluation and are thus well-suited for comparing model outcomes.

4.3. Experiment setting

Our experimental framework evaluated the performance impact of CTIP on two generative models: DF-GAN and LDM. We also conducted a comparative experiment between CTIP and two pre-training methods: CLIP and Tabularbinning [15]. Tabularbinning (TB) is a pre-training method designed for auto-encoding-based self-supervised learning, which utilizes only tabular data. CLIP converts tabular data into text and pre-trains using images, with CLIP loss applied to an imputation generated by nomic-BERT [19]. Therefore, we compare a pre-training method that uses both tabular data and images multimodally (CTIP), with a method that uses only tabular data (TB), and with a method that converts tabular data into text (CLIP). Additionally, in cases without Pre-train (w/o Pre-train), the encoder responsible for generating condition embeddings is used without pre-training. Additional implementation details and hyperparameter settings can be found in the supplementary material.

4.4. Experiment Results

Table 1 shows the results of DF-GAN and LDM when employing CTIP or CLIP, using text and tabular data, respectively. The results demonstrate superior performance when using tabular data with a tabular encoder instead of converting tabular data to text. This finding is due to the fact that, although embeddings are created with a natural language model by converting the tabular data into text, the numerical information inherent in the tabular data is not well captured. Slight differences in specific values can significantly influence the accuracy of the actual tire footprint image, but this sensitive learning is constrained when using a natural language model. While we observed some improvement in performance with pre-training using CLIP, it was still inferior to that achieved using a tabular encoder, highlighting the limitations of converting tabular data to text and processing it through natural language models.

Furthermore, if we compare the results of CTIP and TB, we can see that TB pre-trained using only tabular data performs worse than CTIP. The outcome highlights the importance of integrating image and tabular information with CTIP. We also found that CTIP enhances performance in both DF-GAN and LDM, with notable improvements in performance and similarity metrics, mainly showing the best results with LDM when using CTIP.

The scatter plot in Figure 3 further illustrates improved predicted values and performance metrics for DF-GAN and LDM. It is evident that the scatter plot in orange color, representing the results after applying CTIP, clusters more

Table 1. Tire performance & Image similarity result

Data Type	Models	Performance			Similarity		
		CL↓	CW↓	CA↓	LPIPS↓	SSIM↑	PSNR↑
Text	DF-GAN w/o Pre-train	27.86%	16.28%	40.11%	0.3093	0.7022	13.374
	DF-GAN with CLIP	20.26%	11.79%	28.04%	0.2478	0.7392	15.102
	LDM w/o Pre-train	30.01%	17.62%	47.49%	0.3279	0.6767	12.952
	LDM with CLIP	13.24%	7.78%	19.03%	0.2019	0.7534	16.045
Tabular	DF-GAN w/o Pre-train	16.96%	7.50%	25.18%	0.1857	0.7638	16.357
	DF-GAN with TB	12.80%	6.82%	17.28%	0.1629	0.7689	16.645
	DF-GAN with CTIP	5.83%	3.86%	8.02%	0.1291	0.7777	17.104
	LDM w/o Pre-train	9.79%	5.52%	15.67%	0.1521	0.7674	16.668
	LDM with TB	8.12%	5.15%	12.08%	0.1406	0.7320	16.975
	LDM with CTIP	**3.29%**	**3.65%**	**5.55%**	**0.1154**	**0.7891**	**17.666**

Table 2. Tire performance results for feature few-short and zero-shot

Data Type	Models	Few-shot			Zero-shot		
		CL↓	CW↓	CA↓	CL↓	CW↓	CA↓
Text	DF-GAN w/o Pre-train	28.64%	16.64%	39.82%	28.45%	16.48%	41.90%
	DF-GAN with CLIP	24.55%	15.90%	30.87%	27.40%	15.24%	37.03%
	LDM w/o Pre-train	32.88%	18.17%	50.46%	24.96%	18.22%	39.70%
	LDM with CLIP	17.86%	9.60%	24.69%	18.94%	13.49%	27.53%
Tabular	DF-GAN w/o Pre-train	21.46%	10.20%	30.96%	26.11%	7.79%	27.00%
	DF-GAN with TB	18.64%	8.75%	29.82%	28.45%	8.48%	24.12%
	DF-GAN with CTIP	4.30%	4.26%	7.44%	13.88%	8.39%	18.85%
	LDM w/o Pre-train	12.66%	6.38%	20.43%	19.47%	8.68%	30.99%
	LDM with TB	10.12%	6.08%	18.41%	16.19%	6.12%	24.11%
	LDM with CTIP	**3.73%**	**4.46%**	**7.18%**	**6.51%**	**5.69%**	**7.64%**

closely around the center line compared to the blue scatter plot without CTIP. Overall, CTIP significantly improved the quality of the images generated, and in all cases, the results were more favorable than processing tabular data with text using CLIP. Results for the generated images can be found in the supplementary material.

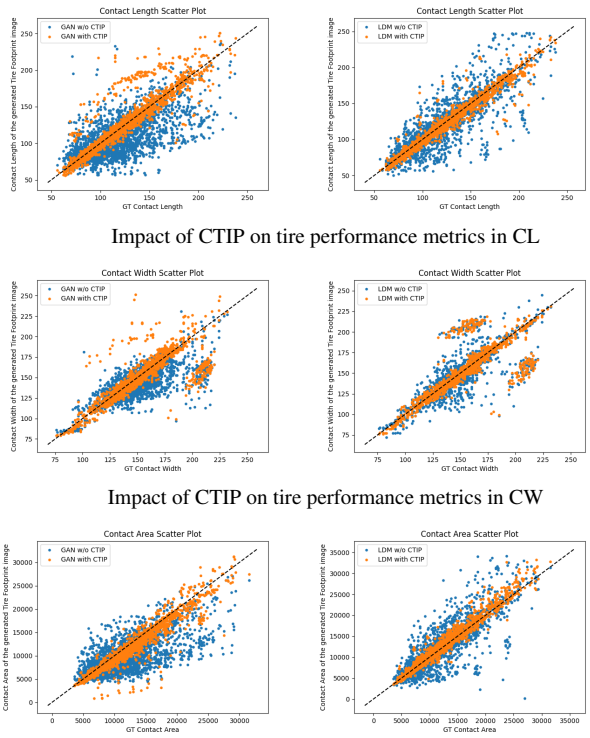
5. Analysis

5.1. Feature Few & Zero-shot Results

In the manufacturing industry, particularly in tire development, encountering new features such as tire patterns is a common challenge. In this study, we define feature few-shot scenarios as cases where a specific tire pattern is present in the training set five times or less and feature zero-shot scenarios as cases where a tire pattern is not present in the training set at all. These scenarios represent significant hurdles in generating accurate tire footprint images. Table 2 shows the impact of CTIP on these challenges, particularly in the feature few-shot and zero-shot scenarios.

Without CTIP, the DF-GAN and LDM models perform poorly in feature few-shot and zero-shot scenarios. These results are significantly worse than those presented in Table 1, highlighting the difficulty of accurately generating tire footprint images under these constraints. However, applying CTIP leads to a substantial improvement, clearly showing that the models struggle to generate robust images when trained solely on the image generation model but produce much more reliable images when CTIP is applied. These improvements underscore CTIP's effectiveness in generating accurate tire footprint images, even in challenging scenarios like feature few-shot and zero-shot.

An essential aspect of tire footprint generation is avoiding producing incorrect images. In the zero-shot scenario where the tire pattern is unknown, models without CTIP generate incorrect patterns randomly. However, with CTIP, the models generate a similar overall outline but do not explicitly depict the unknown tire pattern, demonstrating that CTIP effectively learns how specific features in the tabular data influence particular image parts. The resulting images for feature few-shot and zero-shots can be found in the supplemental material.



Impact of CTIP on tire performance metrics in CL

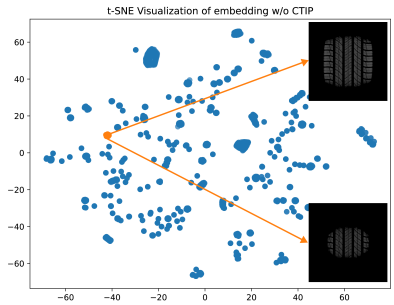
Impact of CTIP on tire performance metrics in CW

Impact of CTIP on tire performance metrics in CA

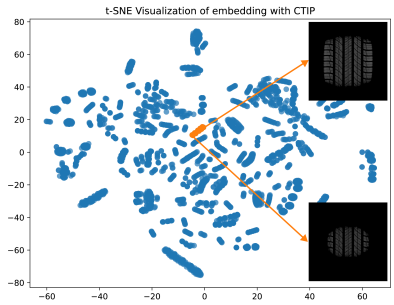
Figure 3. The plot for DF-GAN is shown on the left and the plot for LDM on the right. The vertical axis is the value of the generation image and the horizontal axis is the value of the ground truth. The blue color represents the values without CTIP and the orange color represents the values with CTIP.

5.2. Impact of Test Condition

Further analysis and a case study utilizing t-SNE plots [28] provide a clearer picture of the impact of CTIP. In Figure 4, we can see that without CTIP, cases with the same specification but different test conditions are plotted similarly. However, with CTIP, we can see that similar embeddings occur for the same tire specification, but distinct embeddings emerge as the test conditions change. Specifically, in Table 3, we can see that the embeddings change consistently as the test conditions change, implying that CTIP learns the differences in the images as the test conditions change, producing distinct embeddings. Additionally, the sensitivity to the test conditions can be seen in Figure 5. Without CTIP, the model fails to accurately capture the test condition information, resulting in tire footprint images of similar size across all three test conditions. It indicates that the model struggles to reflect the changes caused by the different test conditions. On the other hand, when CTIP is applied, the generated tire footprint images reflect the differences caused by each test condition, demonstrating the effectiveness of CTIP in capturing and representing detailed



t-SNE visualization of condition embeddings without CTIP



t-SNE visualization of condition embeddings with CTIP

Figure 4. Visualization of condition embedding with and without CTIP using t-SNE. Also, the tire footprint image is shown for the same tire specification in orange.

Table 3. Embedding changes based on test conditions in t-SNE

Inflation air	Test Load	X(CTIP)	Y(CTIP)	X(w/o CTIP)	Y(w/o CTIP)
2.46	360.0	-4.58	10.51	-42.39	8.99
2.46	480.0	-3.57	11.69	-41.65	9.00
2.46	600.0	-2.36	13.21	-41.47	9.63
2.46	720.0	-1.43	14.39	-41.95	10.09
2.46	840.0	-0.62	15.42	-42.60	9.74

feature variations. This result applies not only to the test conditions but also to the different specification features that affect the tire footprint images.

In sum, this analysis shows that image generation models trained with a single pair often have limitations in learning the impact of changes in specific features on the image. However, CTIP, which is trained with multiple pairs, successfully learns and adapts to the impact of changes in specific features on the image. This capability underscores the advanced functionality of CTIP in handling complex, variable data inputs effectively.

5.3. Evaluating CTIP through Image Similarity

To evaluate the quality of the embedding, it should be checked whether the tabular data is embedded similarly to the similarity of the GT images. For example, similar tire footprint images should be embedded similarly. Therefore, the image similarity between the GT images and the Eu-

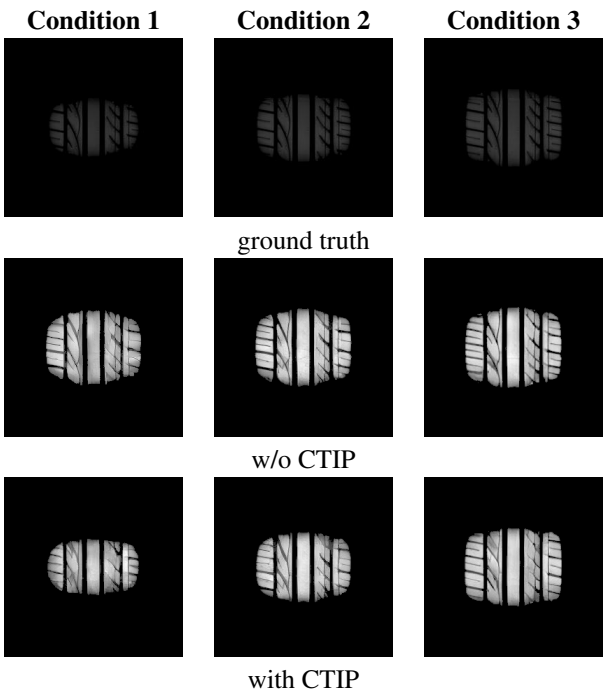


Figure 5. It shows the change of the tire footprint according to the Test Condition.

clidean distance between the condition embeddings should align to be similar. Therefore, we compared the Euclidean distance between the condition embeddings with and without CTIP to the similarity between the GT images to see the impact of CTIP on the condition embeddings.

First, we generated a similarity matrix for the tire footprint images in the test set using LPIPS as an image similarity metric. Then, for each condition embedding, a matrix to calculate the Euclidean distance between embeddings to compare the similarity of embeddings without and with CTIP was generated:

$$S_{\text{LPIPS}} = \text{LPIPS}(I_{\text{gt}}, I_{\text{gt}}) \tag{1}$$

$$D_{\text{w}\backslash\text{o CTIP}} = \text{ED}(C_{\text{w}\backslash\text{o CTIP}}, C_{\text{w}\backslash\text{o CTIP}}) \tag{2}$$

$$D_{\text{with CTIP}} = \text{ED}(C_{\text{with CTIP}}, C_{\text{with CTIP}}) \tag{3}$$

, where I_{gt} represents the GT tire footprint images and $C_{\text{w}\backslash\text{o CTIP}}$, $C_{\text{with CTIP}}$ represents the condition embedding without and with CTIP.

We then computed the cosine similarity of the similarity matrix S_{LPIPS} of the generated tire footprint image to the Euclidean distance matrix of the condition embedding $D_{\text{w}\backslash\text{o CTIP}}$ and $D_{\text{with CTIP}}$, respectively:

$$CosineSim(S_{\text{LPIPS}}, D_{\text{w}\backslash\text{o CTIP}}) : 0.8832 \tag{4}$$

$$CosineSim(S_{\text{LPIPS}}, D_{\text{with CTIP}}) : 0.9365 \tag{5}$$

The application of CTIP significantly enhanced the similarity of condition embeddings to the real tire footprint images, as indicated by the cosine similarity scores. These results confirm that CTIP is able to generate condition embeddings with embeddings that more closely resemble the actual images and their embeddings. Therefore, CTIP can generate better-quality embeddings, which is critical for generating the images we target. The pseudo code for the experiments can be found in the supplementary material.

6. Limitation and Future Work

The limitation of this study is that we used only a single dataset to validate CTIP due to the lack of comparable open datasets in this specific context. The tire footprint dataset contains over 60,000 pairs of tabular data and images. The tabular data is not tabularized with information about the paired images but features about the tire and the test condition. Additionally, the features in the tabular data are mostly interdependent rather than independent. Because we have not found any open benchmark datasets of this kind, we could not validate CTIP across multiple domains. However, the novel tabular-to-image approach proposed with CTIP may inspire further discoveries in the industry and encourage the release of relevant datasets. Future research should explore the potential of applying CTIP to tabular-to-image conversion tasks across a broader range of domains.

7. Conclusion

This research highlights the significance of a proposed tabular-to-image method for generating target images from the tabular data found in the manufacturing industry, empirically demonstrating that CTIP can enhance the quality of generated images. CTIP not only improves the overall quality of image generation but also produces images that are robust in scenarios characterized by sparse feature information in the training set, such as in feature few-shot and zero-shot cases. We validated these results with real-world industry metrics and image similarity metrics. Additionally, we observed that the embedding of tabular data using CTIP closely aligns with the similarity of the ground truth image and exhibits increased sensitivity to the testing conditions. This research not only serves as an example of the practical application of image generation models in industrial settings, particularly in the manufacturing sector, but also emphasizes the role of integrating tabular data into the generative modeling process.

8. Acknowlegement

This work was supported by Hankook Tire & Technology Co., Ltd, Republic of Korea, and some experiments were conducted using the instruments of Hankook Tire & Technology Co., Ltd. The authors extend their sincere thanks to Hankook Tire & Technology Co., Ltd for their help and support (Project No. G01230582).

References

[1] Han Fang, Pengfei Xiong, Luhui Xu, and Yu Chen. Clip2video: Mastering video-text retrieval via image clip. *arXiv preprint arXiv:2106.11097*, 2021. 2

[2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 2

[3] Andrey Guzhov, Federico Raue, Jörn Hees, and Andreas Dengel. Audioclip: Extending clip to image, text and audio. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 976–980. IEEE, 2022. 2

[4] Paul Hager, Martin J Menten, and Daniel Rueckert. Best of both worlds: Multimodal contrastive learning with tabular and imaging data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23924–23935, 2023. 3

[5] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 5

[6] Alain Hore and Djemel Ziou. Image quality metrics: Psnr vs. ssim. In *2010 20th international conference on pattern recognition*, pages 2366–2369. IEEE, 2010. 5

[7] Weichen Huang. Multimodal contrastive learning and tabular attention for automated alzheimer's disease prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2473–2482, 2023. 3

[8] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks. *Advances in neural information processing systems*, 34:852–863, 2021. 3

[9] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4401–4410, 2019. 3

[10] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8110–8119, 2020. 3

[11] Jonghun Kim and Hyunjin Park. Adaptive latent diffusion model for 3d medical image to image translation: Multimodal magnetic resonance imaging study. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 7604–7613, 2024. 1

[12] Min Woo Kim and Nam Ik Cho. Whfl: Wavelet-domain high frequency loss for sketch-to-image translation. In *Proceedings of the IEEE/CVF Winter Conference on applications of computer vision*, pages 744–754, 2023. 1

[13] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 3

[14] Gihyun Kwon and Jong Chul Ye. Clipstyler: Image style transfer with a single text condition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18062–18071, 2022. 2

[15] Kyungeun Lee, Ye Seul Sim, Hye-Seung Cho, Moonjung Eo, Suhee Yoon, Sanghyu Yoon, and Woohyung Lim. Binning as a pretext task: Improving self-supervised learning in tabular domains. *arXiv preprint arXiv:2405.07414*, 2024. 5

[16] Ziyi Lin, Shijie Geng, Renrui Zhang, Peng Gao, Gerard de Melo, Xiaogang Wang, Jifeng Dai, Yu Qiao, and Hongsheng Li. Frozen clip models are efficient video learners. In *European Conference on Computer Vision*, pages 388–404. Springer, 2022. 2

[17] Adyasha Maharana, Darryl Hannan, and Mohit Bansal. Storydall-e: Adapting pretrained text-to-image transformers for story continuation. In *European Conference on Computer Vision*, pages 70–87. Springer, 2022. 1

[18] Tanvir Mahmud and Diana Marculescu. Ave-clip: Audioclip-based multi-window temporal transformer for audio visual event localization. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 5158–5167, 2023. 2

[19] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder, 2024. 5

[20] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015. 3

[21] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 3, 5

[22] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, pages 234–241. Springer, 2015. 5

[23] Aditya Sanghi, Hang Chu, Joseph G Lambourne, Ye Wang, Chin-Yi Cheng, Marco Fumero, and Kamal Rahimi Malekshan. Clip-forge: Towards zero-shot text-to-shape generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18603–18613, 2022. 2

[24] Saksham Suri, Moustafa Meshry, Larry S Davis, and Abhinav Shrivastava. Grit: Gan residuals for paired image-to-image translation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 4965–4975, 2024. 1

[25] Alex Tang, Gromit Chan, Ryan A Rossi, Chang Xiao, and Eunyee Koh. Tabular data to image generation: Benchmark data, approaches, and evaluation. In *2023 IEEE International Conference on Big Data (BigData)*, pages 1539–1543. IEEE, 2023. 2

[26] Ming Tao, Bing-Kun Bao, Hao Tang, and Changsheng Xu. Galip: Generative adversarial clips for text-to-image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14214–14223, 2023. 2

[27] Ming Tao, Hao Tang, Fei Wu, Xiao-Yuan Jing, Bing-Kun Bao, and Changsheng Xu. Df-gan: A simple and effective baseline for text-to-image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16515–16525, 2022. 1, 3, 4

[28] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008. 7

[29] Yael Vinker, Yuval Alaluf, Daniel Cohen-Or, and Ariel Shamir. Clipascene: Scene sketching with different types and levels of abstraction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4146–4156, 2023. 2

[30] Yael Vinker, Ehsan Pajouheshgar, Jessica Y Bo, Roman Christian Bachmann, Amit Haim Bermano, Daniel Cohen-Or, Amir Zamir, and Ariel Shamir. Clipasso: Semantically-aware object sketching. *ACM Transactions on Graphics (TOG)*, 41(4):1–11, 2022. 2

[31] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004. 5

[32] Tianyi Wei, Dongdong Chen, Wenbo Zhou, Jing Liao, Zhentao Tan, Lu Yuan, Weiming Zhang, and Nenghai Yu. Hairclip: Design your hair by text and reference image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18072–18081, 2022. 2

[33] Ho-Hsiang Wu, Prem Seetharaman, Kundan Kumar, and Juan Pablo Bello. Wav2clip: Learning robust audio representations from clip. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4563–4567. IEEE, 2022. 2

[34] Hu Xu, Gargi Ghosh, Po-Yao Huang, Dmytro Okhonko, Armen Aghajanyan, Florian Metze, Luke Zettlemoyer, and Christoph Feichtenhofer. Videoclip: Contrastive pre-training for zero-shot video-text understanding. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6787–6800, 2021. 2

[35] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. Imagereward: Learning and evaluating human preferences for text-to-image generation. *Advances in Neural Information Processing Systems*, 36, 2024. 1

[36] Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models: A comprehensive survey of methods and applications. *ACM Computing Surveys*, 56(4):1–39, 2023. 2

[37] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3836–3847, 2023. 1

[38] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. 5

[39] Yufan Zhou, Ruiyi Zhang, Changyou Chen, Chunyuan Li, Chris Tensmeyer, Tong Yu, Jiuxiang Gu, Jinhui Xu, and Tong Sun. Towards language-free training for text-to-image generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17907–17917, 2022. 1, 2

A. Dataset Details

A.1. Tire Development Process

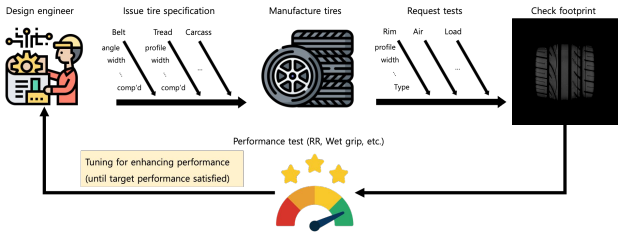


Figure 6. Illustration of the tire development process. An engineer designs tire specifications, and a tire is manufactured to those specifications. For the manufactured tire, photographs are taken of the area where the tire touches the ground under various test conditions, which is called the tire footprint image. Our goal is to generate a tire footprint image directly from the tire specification and test conditions without manufacturing the tire.

In the tire manufacturing industry, the development of new tires is both labor-intensive and resource-demanding. Initially, engineers design a tire by detailing various technical specifications, which guide the creation of experimental prototypes. A crucial step in evaluating these prototypes involves capturing images of the tire's contact patch with the ground, referred to as tire footprints, under diverse conditions. These footprints are essential for assessing the tire's performance and for engineers to visually verify that the prototype matches their design intentions. Often, this evaluation leads to further adjustments in tire specifications. Fig. 6 shows the process for developing a tire.

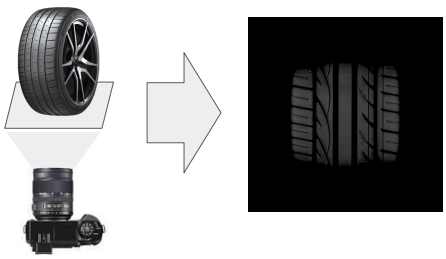


Figure 7. An illustration of tire footprint

Tire Footprint plays a very important role in the tire development process. Tire developers look at the Tire Footprint to not only determine if their design is working as intended, but also to quantify the image to evaluate the performance of the tire. Here are three of the tire performance metrics we used in our experiments. Figure 7 illustrates the description of the tire footprint image, and Figure 8 shows an example of a tire footprint.

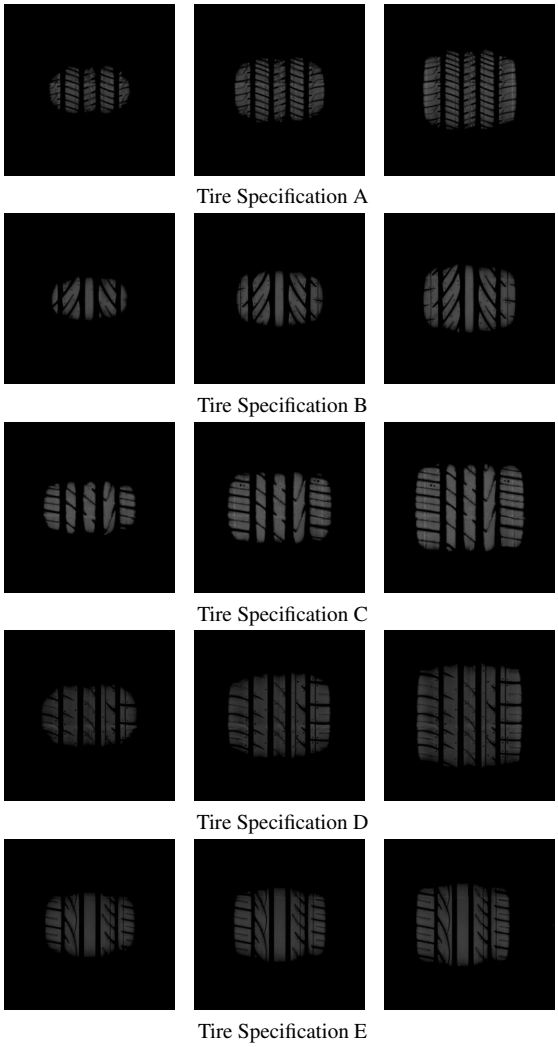


Figure 8. Ground truth images for increasing test load, → in order. As the test load increases, the contact area increases.

A.2. Example of Tabular Data

Table 4 provides examples of tabular data. With 116 features in total, it is impractical to show all of them, so we highlight key examples. For instance, the tire specification features include "Pattern" (a categorical feature) and "Tire Size" (a numeric feature). Additionally, graphical features such as "CTB_Step_Width" and "CTB_Step_Gauge" denote position and value, respectively, forming a graph when connected. Test condition features include "Test_Load" and "Inflation_air," where "Test_Load" indicates the load applied to the tire (affecting the contact area) and "Inflation_air" denotes the tire's air pressure. These two test conditions are the main conditions for tire development, and the ground contact area of the tire will increase or decrease depending on the change in test conditions.

Table 4. Examples of tabular data

Feature Name	Tire Spec or Test Condition	Type	Example
Pattern	Tire Specification	Category	H826
Tire Size	Tire Specification	Numeric	250
...	...	...	...
CTB_Step_Width	Tire Specification	Graph	5.0-3.0-7.0-6.0-...
CTB_Step_Gauge	Tire Specification	Graph	0.0-0.2-0.2-0.2-...
...	...	...	...
Inflation_air	Test Condition	Numeric	3.4
Test_load	Test Condition	Numeric	256

A.3. Example of Graphical Feature

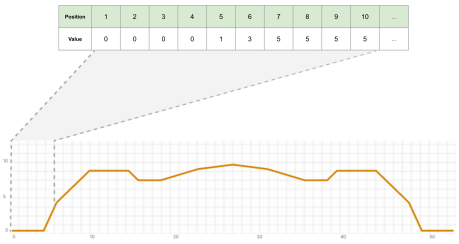


Figure 9. Illustration for graphical feature

A Graphical Feature refers to the structure of a specific part used in a tire. Tabular data is recorded in two categories: position and value. We visualize it and generate an embedding using a CNN. This data preprocessing approach demonstrates the scalability of tabular-to-image conversion for multi-modal applications. Figure 9 illustrates the example of graphical feature.

B. Implementation Details

B.1. Latent Diffusion U-net

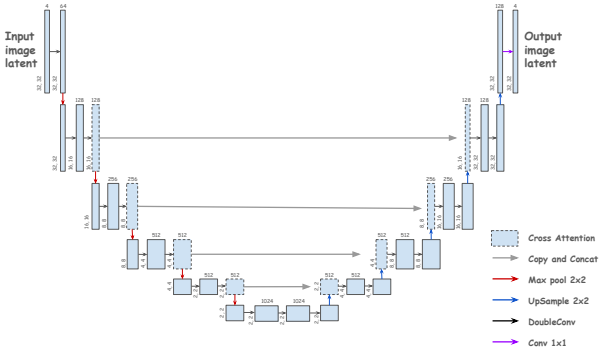


Figure 10. Illustration of the U-net structure.

The structure of the U-net we use is depicted in Figure 10. It consists of five downsampling and five upsampling steps, projecting the embedding into latent space using a VAE. In the encoder part, which performs the downsampling, we apply cross-attention on the embedding before downsampling. Similarly, in the decoder part, which handles the upsampling, cross-attention is applied after upsampling. We condition the embedding created by the Tabular encoder by performing cross-attention on a layer of the U-Net.

B.2. Transform Tabular Data into Natural Language

In Experiment 4.4, we also examined the results of using natural language models to convert tabular data into natural language. We transformed the tabular data into natural language, except for graphical features, which we converted into natural language for feature names and feature values. Below are some examples of transforming tabular data into natural language:

Pattern: H825
Tire Size: 250
.
.
Inflation air: 3.4
Test Load: 256

The graphical features were generated as embeddings by a CNN and concatenated with embeddings generated by a natural language model. This experiment involved replacing the tabular encoder in the CTIP framework with an encoder that solely utilizes a natural language model.

B.3. Hyperparameter Settings

First, we pre-train on CTIP. CTIP will be trained with a total of 30 epochs and will only be trained with the train dataset. We freeze the weights of the tabular encoder on the pre-trained CTIP and then train DF-GAN and LDM on the pre-trained CTIP. DF-GAN is trained for a total of 100 epochs, and LDM is trained for a total of 1000 epochs. Relatively speaking, LDM is faster to train, taking about the same amount of time (36 hours). In the case of LDM, VAE

Table 5. CTIP hyperparameter setting

	CTIP	DF-GAN	LDM
Batch size	128	128	128
Embedding dimension	256	256	256
Learning rate	0.001	0.0001	0.0001
Training epochs	30	100	1000
Adam β_1	0.0	0.0	0.0
Adam β_2	0.9	0.9	0.9
Temperature t	0.01	-	-
ViT layers	4	-	-
ViT heads	4	-	-
ViT patch size	4	-	-

imports and uses pre-trained weights.

C. Experiment Details

C.1. Tire Performance Metrics

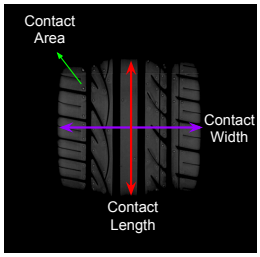


Figure 11. Illustration of tire performance metrics.

The Tire Performance metrics CL, CW, and CA are calculated based on the contact area of the tire. To measure these, we first remove the background of the generated image, retaining only the tire footprint—defined as the area where the tire has made contact—and set the remaining pixel values to zero. CL is determined by measuring the distance from the topmost to the bottommost non-zero pixel within the tire footprint. CW is measured from the leftmost to the rightmost non-zero pixel within the tire footprint. CA is calculated as the total number of non-zero pixels, representing the area touched by the tire. These metrics are crucial for evaluating tire performance and are used to rate tires based on specific criteria. Figure 11 shows an example of a tire performance metric.

C.2. Image Results

Figures 12 and 13 showcase the results of experiments using a tabular encoder and a BERT encoder, respectively. These figures display the outcomes for three test conditions with two tire specifications and the resulting images generated by both DF-GAN and LDM with and without pre-trained models. Overall, the tabular encoder produced more accurate images compared to using BERT with text conversion of the tabular data. Although pre-training with CLIP after generating embeddings with BERT by converting the tabular data to text led to more accurate images, they were not as precise as those produced by pre-training the tabular encoder with CTIP.

On the other hand, using a tabular encoder and pre-training it with CTIP resulted in nearly accurate tire footprints. Improvements were observed for both DF-GAN and LDM, with LDM showing the best performance when pre-trained with CTIP. It is evident that this approach accurately captures the geometry of the tire footprint, which is crucial for reliable image generation.

C.3. Few-shot and Zero-shot Image Results

Figure 14 shows the result images for feature few-shot and zero-shot. Without CTIP, the generated images look significantly different from the GT images and tend to randomly produce certain patterns, which is a poor result as it generates incorrect images. In contrast, when CTIP is used, the tire footprint shape is generated more accurately. Notably, in the first pattern case, feature zero-shot, the overall shape is roughly correct. Additionally, with CTIP, unseen patterns appear blurred rather than being randomly generated. These results demonstrate that CTIP is effective in generating robust images, even in challenging cases.

C.4. Pesudo Code for Evaluating CTIP through Image Similarity

Algorithm 1 presents the pseudocode for the experiment detailed in Analysis 5.3. To validate that CTIP indeed produces high-quality embeddings, we conducted an experiment to determine if the similarity between real images and the embeddings from a tabular encoder pre-trained with CTIP are closely aligned. Initially, we compute the LPIPS (Learned Perceptual Image Patch Similarity) between the ground truth images of the test set to create an $N \times N$ matrix S_{LPIPS} (steps 3-11 in Algorithm 1). Subsequently, we generate embeddings for the test set from both the CTIP-trained and untrained tabular encoders. We then measure the Euclidean distance between these embeddings to create the $N \times N$ matrices $D_{\text{with CTIP}}$ and $D_{\text{without CTIP}}$ (steps 12-23 in Algorithm 1). Finally, we compute the cosine similarity of $D_{\text{with CTIP}}$ and $D_{\text{without CTIP}}$ against S_{LPIPS}. By comparing the two cosine similarities, we analyze the results as shown in Analysis 5.3.

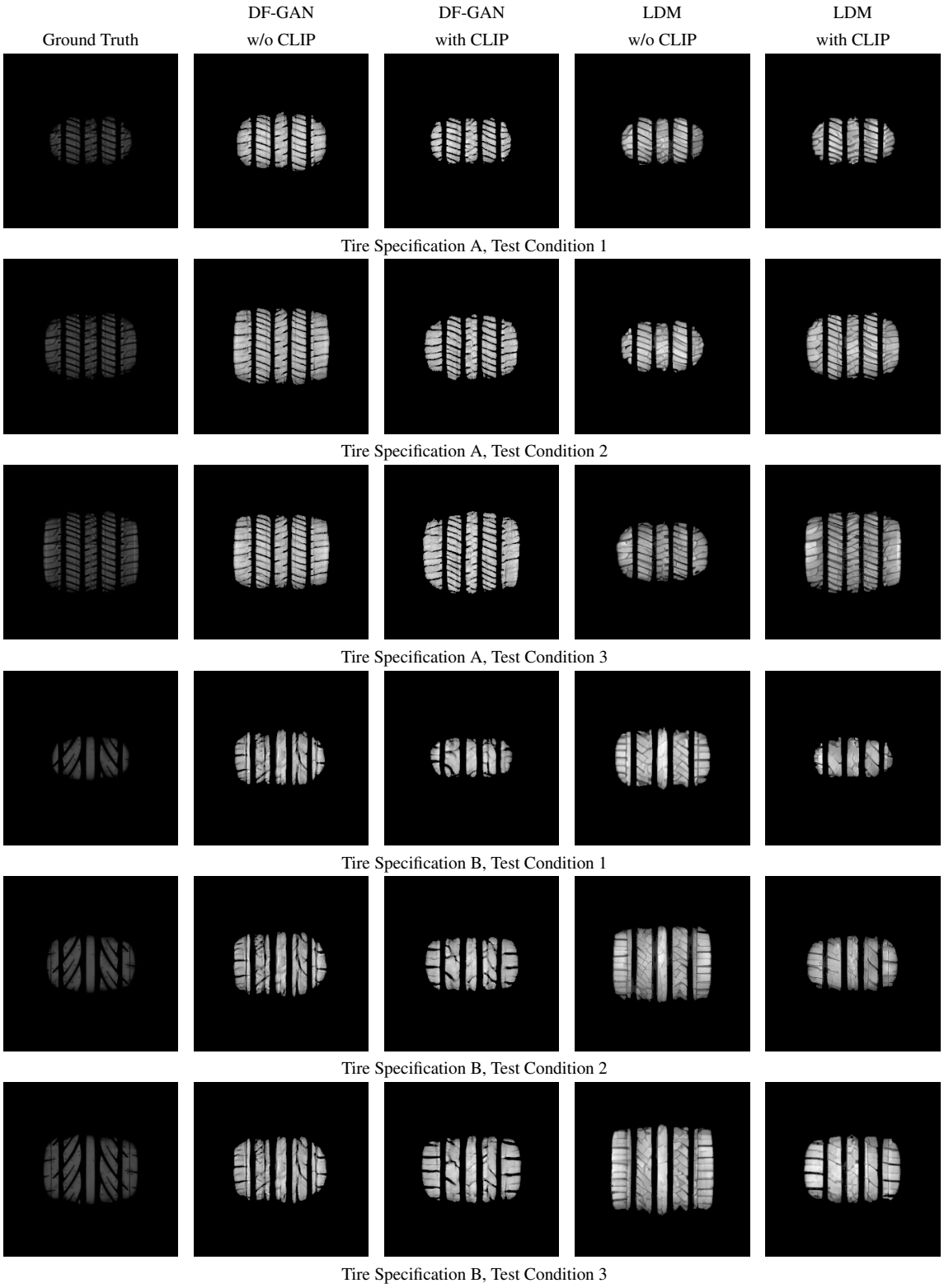


Figure 12. Examples of tire footprint images for three test conditions using CTIP.

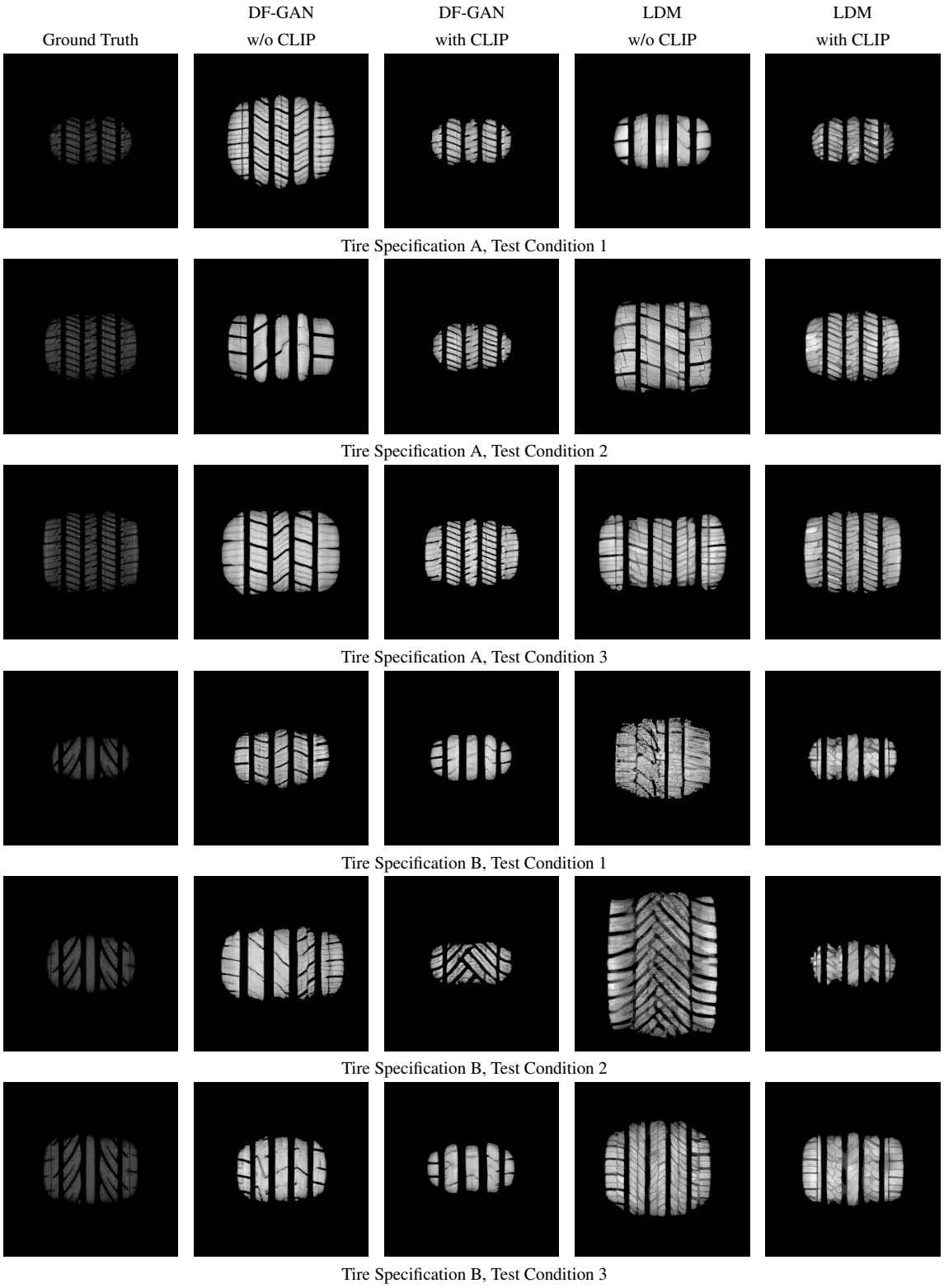


Figure 13. Examples of tire footprint images for three test conditions using clip.

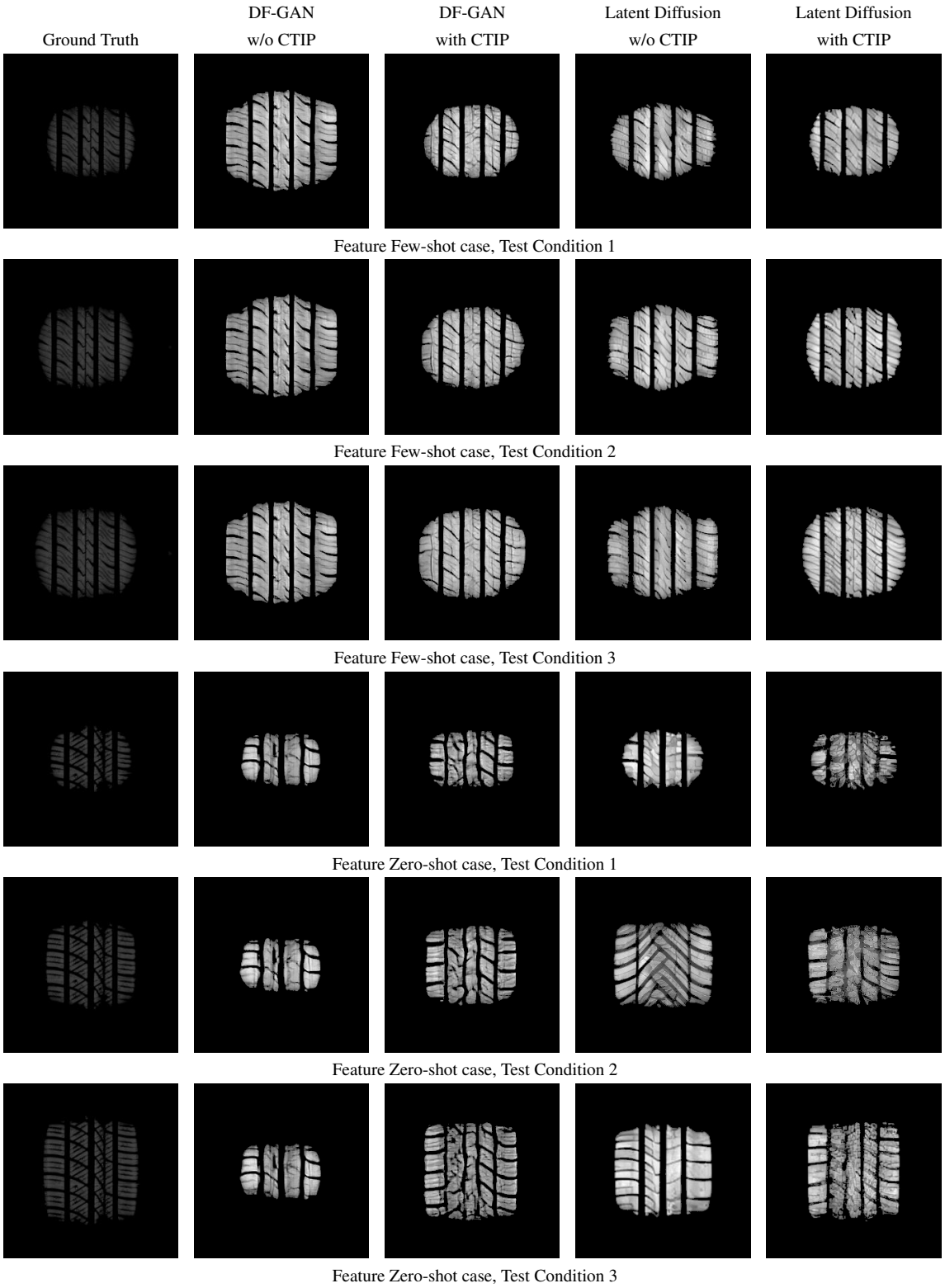


Figure 14. Examples of the feature few-shot & zero-shot cases, where there is under five tire specifications with the same tire pattern in the train dataset. The zero-shot case is that there are no tire specifications with the same tire pattern in the train dataset.

Algorithm 1 Pesudo code for Evaluating CTIP through Image Similarity

```
Input: gt images I_gt, condition embeddings with CTIP E_with CTIP, condition embeddings without CTIP E_without CTIP
Output: Cosine Similarity
procedure IMAGEMATRIX(I_gt)
    N ← Numer of test sets
    Initialize S_LPIPS as a N × N matrix
    for each pair of images in I_gt do
        Compute LPIPS similarity
        Update S_LPIPS
    end for
    return S_LPIPS
end procedure
procedure DISTANCEMATRIX(C_with CTIP, C_without CTIP)
    N ← Numer of test sets
    Initialize D_with CTIP as a N × N matrix
    Initialize D_without CTIP as a N × N matrix
    for C_with CTIP, C_without CTIP do
        for each pair of embeddings do
            Calculate Euclidean distance
            Update D_with CTIP or D_without CTIP
        end for
    end for
    return D_with CTIP, D_without CTIP
end procedure
procedure COMPUTE COSINE SIMILARITY(D_with CTIP, D_without CTIP, S_LPIPS)
    Calculate cosine similarity between S_LPIPS and D_with CTIP
    Calculate cosine similarity between S_LPIPS and D_without CTIP
end procedure
S_LPIPS ← IMAGEMATRIX(I_gt)
D_with CTIP, D_without CTIP ← DISTANCEMATRIX(C_with CTIP, C_without CTIP)
COMPUTECOSINESIMILARITY(D_with CTIP, D_without CTIP, S_LPIPS)
```